

RUGGINE

Applicazione di Chat Client/Server

con gruppi su invito, log di sistema e portabilità multi-piattaforma

Agenda

01 – 03

Progetto & Architettura

Obiettivi e requisiti
Workspace Cargo
Architettura del sistema

04 – 08

Protocollo & Comunicazione

Protocollo di comunicazione
Registrazione e login
Flusso di un messaggio
Gruppi su invito · Messaggi diretti

09 – 14

Implementazione

Concorrenza: Tokio e async/await
Stato condiviso: Arc e Mutex
Canali mpsc · Database SQLite
Server HTTP Axum · GUI egui/eframe

15 – 19

Qualità & Risultati

Logging CPU
Ottimizzazioni build release
Portabilità multi-piattaforma
Dimensione eseguibili · Conclusioni

Obiettivi e Requisiti

Requisiti Funzionali

- Chat testuale client/server in tempo reale
- Iscrizione al primo avvio (username univoco)
- Login automatico per utenti esistenti
- Creazione e gestione di gruppi
- Ingresso ai gruppi solo su invito esplicito
- Solo i membri possono invitare altri
- Messaggi diretti (DM) tra utenti

Requisiti Non Funzionali

- Portabilità: ≥ 2 piattaforme (Windows, Linux, macOS)
- Basso consumo CPU e RAM in idle
- Eseguitibile compatto (< 5 MB)
- Log CPU ogni 2 minuti su file `cpu.log`
- Gestione errori senza `unwrap()`
- Nessuna dipendenza da runtime esterno

Workspace Cargo: struttura del progetto

Struttura directory

```
ruggine/  
├── Cargo.toml  
├── ruggine_server/  
│   ├── src/main.rs  
│   └── Cargo.toml  
├── ruggine_client/  
│   ├── src/main.rs  
│   └── Cargo.toml  
└── ruggine_common/  
    ├── src/lib.rs  
    └── Cargo.toml
```

← workspace

ruggine_server

Server WebSocket (porta 4000)
Server HTTP Axum (porta 3000)
Database SQLite embedded
Logging CPU ogni 120s

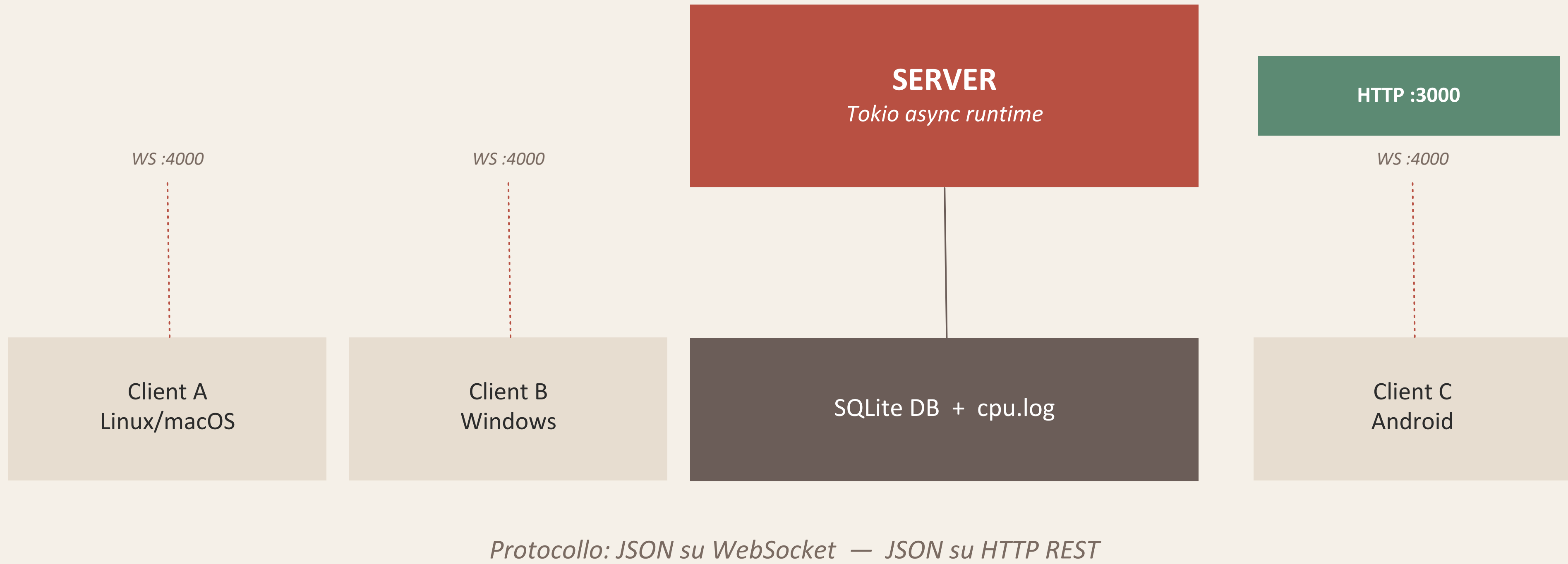
ruggine_client

GUI nativa con egui/eframe
Cross-platform (Win/Linux/macOS)
Comunicazione asincrona via mpsc
Thread UI separato dal thread I/O

ruggine_common

ClientMessage / ServerMessage
GroupInfo, UserDTO
Serde per serializzazione JSON
Condiviso da server e client

Architettura del Sistema



Protocollo di Comunicazione

ClientMessage (client → server)

```
Register { username }
SendMessage { group_id, content }
CreateGroup { name }
InviteUser { group, username }
AcceptInvite { group }
RejectInvite { group }
LeaveGroup { group }
DirectMessage { to_username, content }
InviteToGroup { username_to_invite,
group_id }
JoinGroup { group_id }
ListGroups
```

ServerMessage (server → client)

```
RegisterResponse {success, reason}
Error {message}
NewMessage {group_id, sender_username, content,
timestamp}
InviteReceived {group_id, group_name,
inviter_username}
UserJoinedGroup {group_id, username}
GroupCreated {id, name}
InviteSent {group, username}
JoinedGroup {group_id, group_name}
InviteRejected {group}
GroupList {groups}
LeftGroup {group_id, group_name}
UserLeftGroup {group_id, group_name, username}
DirectMessageReceived {from_username, content,
timestamp}
DirectMessageResult {to_username, success,
reason}
```

Registrazione e Login

1

Avvio client

L'utente inserisce uno username nella schermata di login. Il campo è l'unico dato di autenticazione.

2

ClientMessage::Register

La GUI invia sul canale mpsc il messaggio Register { username }. Il task WebSocket lo serializza in JSON e lo trasmette al server.

3

Server: query SQLite

SELECT id FROM users WHERE username = ?
→ Se non esiste: INSERT INTO users → nuovo utente
→ Se esiste: login automatico (bentornato!)

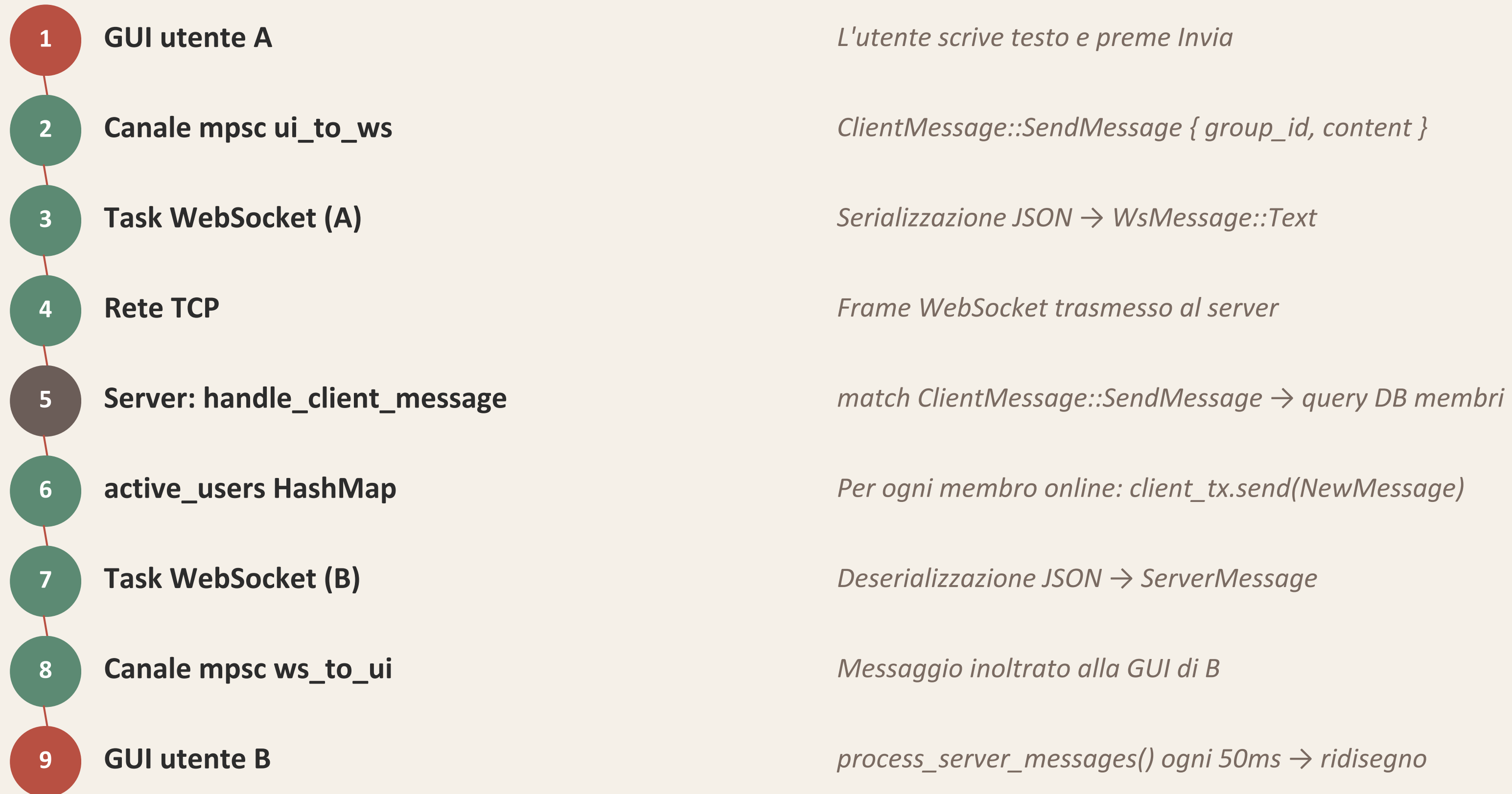
4

ServerMessage::RegisterResponse

Il server risponde con { success: true/false, reason }. Il client aggiorna la mappa active_users con il Sender del canale mpsc.

⚠ Nessuna password: scelta semplificativa per il progetto. In produzione si userebbe bcrypt + JWT.

Flusso di un Messaggio



Gruppi e Sistema di Inviti

`/create <nome> → CreateGroup`

1. Server fa INSERT INTO groups
2. Inserisce creatore in group_members
3. Risponde con GroupCreated { id, name }
4. Client aggiorna la lista gruppi

`/accept <gruppo> → AcceptInvite`

1. DELETE FROM group_invites
2. INSERT INTO group_members
3. Broadcast UserJoinedGroup a tutti i membri
4. Il nuovo membro vede il gruppo in lista

`/invite <user> <gruppo> → InviteUser`

1. Server verifica che l'invitante sia membro
2. INSERT INTO group_invites (pending)
3. Se utente online: InviteReceived { group_id, group_name, inviter_username }
4. L'invitato vede notifica nella GUI + risposta al mittente: InviteSent { group, username }

`/reject <gruppo> → RejectInvite`

1. DELETE FROM group_invites
2. Risponde InviteRejected { group }
3. Nessuna notifica agli altri membri

⚠ Solo membri possono invitare altri utenti

Messaggi Diretti (DM)

1

Comando /dm

L'utente scrive /dm <username> <testo>. Il client invia `ClientMessage::DirectMessage { to_username, content }`.

2

Verifica destinatario

Il server fa `SELECT COUNT(*) FROM users WHERE username = ?` — verifica che l'utente esista nel DB, anche se offline.

3

Utente online?

Cerca il destinatario in `active_users` (`HashMap<username, Sender>`).

→ Trovato: invia `ServerMessage::DirectMessage { from_username, content }`

→ Non trovato: risponde con `DirectMessageResult { success: false }`

4

Ricezione

La GUI del destinatario mostra il messaggio con etichetta 'DM da X', distinto visivamente dai messaggi di gruppo.

5

Conferma mittente

Il server risponde al mittente con `DirectMessageResult { success: true/false, reason }`. Se fallisce, la GUI rimuove il messaggio ottimisticamente aggiunto.

Concorrenza: Tokio e async/await

Il problema

Rust standard è sincrono: aspettare dati dalla rete blocca tutto il thread. Con 100 client connessi, servirebbero 100 thread del SO — costosi in memoria (stack ~8MB ciascuno).

La soluzione: Tokio

Tokio è un runtime asincrono: usa async/await per sospendere task in attesa di I/O senza bloccare il thread. Un pool di pochi thread reali (uno per core) esegue migliaia di task leggeri.

Nel progetto

Ogni client connesso → 1 task Tokio
3 task permanenti: WebSocket, HTTP, CPU logger

main.rs (server)

```
#[tokio::main]
async fn main() {
    // task CPU logger
    tokio::spawn(async move {
        cpu_logging_task().await;
    });

    // task HTTP Axum :3000
    tokio::spawn(async move {
        axum::serve(listener, app)
            .await.unwrap();
    });

    // loop WebSocket :4000
    loop {
        let (stream, addr) =
            listener.accept().await?;
        tokio::spawn(
            handle_connection(stream, state)
        );
    }
}
```

Stato Condiviso: Arc e Mutex

Arc<AppState> — un server, molti task

Ogni connessione WebSocket viene gestita da un task Tokio separato. Tutti i task devono accedere allo stesso AppState (database + mappa utenti connessi). Arc permette di clonare un puntatore leggero verso lo stato condiviso — senza copiare i dati. L'alternativa con ownership singola renderebbe impossibile gestire più connessioni in parallelo.

Mutex<Connection> — SQLite è single-writer

SQLite non supporta scritture concorrenti. Il Mutex garantisce che al massimo un task alla volta acceda al database, evitando corruzione dei dati. Il rilascio avviene automaticamente a fine scope (RAII) — impossibile dimenticare di sbloccare il lock. La stessa logica si applica alla mappa active_users.

state.rs

```
pub struct AppState {  
    // DB: solo 1 writer alla volta  
    pub db: Mutex<Connection>,  
  
    // Mappa utenti connessi  
    pub active_users:  
        Mutex<HashMap<  
            String,          // username  
            mpsc::Sender      // canale verso client  
        >>,  
}  
  
// Condiviso tra tutti i task:  
pub type SharedState =  
    Arc<AppState>;  
  
// Uso nel codice:  
let db = state.inner.db  
    .lock() // acquisisce il Mutex  
    .map_err(|e| anyhow!(e))?;  
// rilascio automatico a fine scope
```

Canali mpsc

Il problema: due thread, zero memoria condivisa

GUI e task di rete girano su thread separati. Passare dati con memoria condivisa richiederebbe lock su entrambi i lati. I canali mpsc eliminano il problema: ogni thread ha la sua estremità del canale e non tocca mai i dati dell'altro.

Nel client

ui_to_ws: GUI → WebSocket task
(messaggi da inviare al server)

ws_to_ui: WebSocket task → GUI
(messaggi ricevuti dal server)

La GUI controlla ws_to_ui ogni 50ms con
ctx.request_repaint_after()

client/main.rs

```
let (ui_to_ws_tx, ui_to_ws_rx) =
    mpsc::unbounded_channel::<ClientMessage>();
let (ws_to_ui_tx, ws_to_ui_rx) =
    mpsc::unbounded_channel::<ServerMessage>();

// Thread Tokio (I/O di rete)
std::thread::spawn(move || {
    tokio::runtime::Runtime::new()
        .unwrap()
        .block_on(websocket_task(
            ui_to_ws_rx,    // riceve dalla GUI
            ws_to_ui_tx,    // invia alla GUI
        ));
});

// Thread principale: GUI
eframe::run_native("Ruggine",
    options,
    Box::new(|_cc|
        Box::new(RuggineApp::new(
            ui_to_ws_tx,    // invia al WS
            ws_to_ui_rx,    // riceve dal WS
        ))
    ),
);
```


Database SQLite

users

id INTEGER PK
username TEXT UNIQUE

groups

id INTEGER PK
name TEXT

group_members

user_id INTEGER FK
group_id INTEGER FK

group_invites

invited_user_id FK
group_id FK
inviter_user_id FK

Transazioni SQLite per CreateGroup (INSERT groups + INSERT group_members atomici). Prepared statements per query ripetute.

Server HTTP Axum

Cos'è Axum?

Framework web asincrono per Rust, costruito sopra Tokio. Definisce route HTTP associando URL a funzioni handler e risponde in JSON.

Perché HTTP oltre a WebSocket?

Per operazioni occasionali stateless — consultare utenti o gruppi — HTTP è più leggero: una richiesta, una risposta, connessione chiusa. Nessun bisogno di una connessione persistente.

Porta 3000

GET /users → lista utenti
GET /groups → lista gruppi

handlers.rs

```
pub fn build_router(
    state: SharedState
) -> Router {
    Router::new()
        .route("/users",
            get(get_users))
        .route("/groups",
            get(get_groups))
        .with_state(state)
        .layer(CorsLayer::permissive())
}

async fn get_users(
    State(state): State<SharedState>
) -> Json<Vec<UserDTO>> {
    let db = state.db.lock().unwrap();
    // query + serializzazione JSON
    Json(users)
}
```


GUI: egui/eframe

egui + eframe

Libreria GUI immediata (immediate mode): ogni frame l'intera UI viene ridisegnata da zero in base allo stato. Nessun sistema di widget con stato interno nascosto. Con eframe gira su Windows, Linux, macOS nativamente.

Layout dell'app

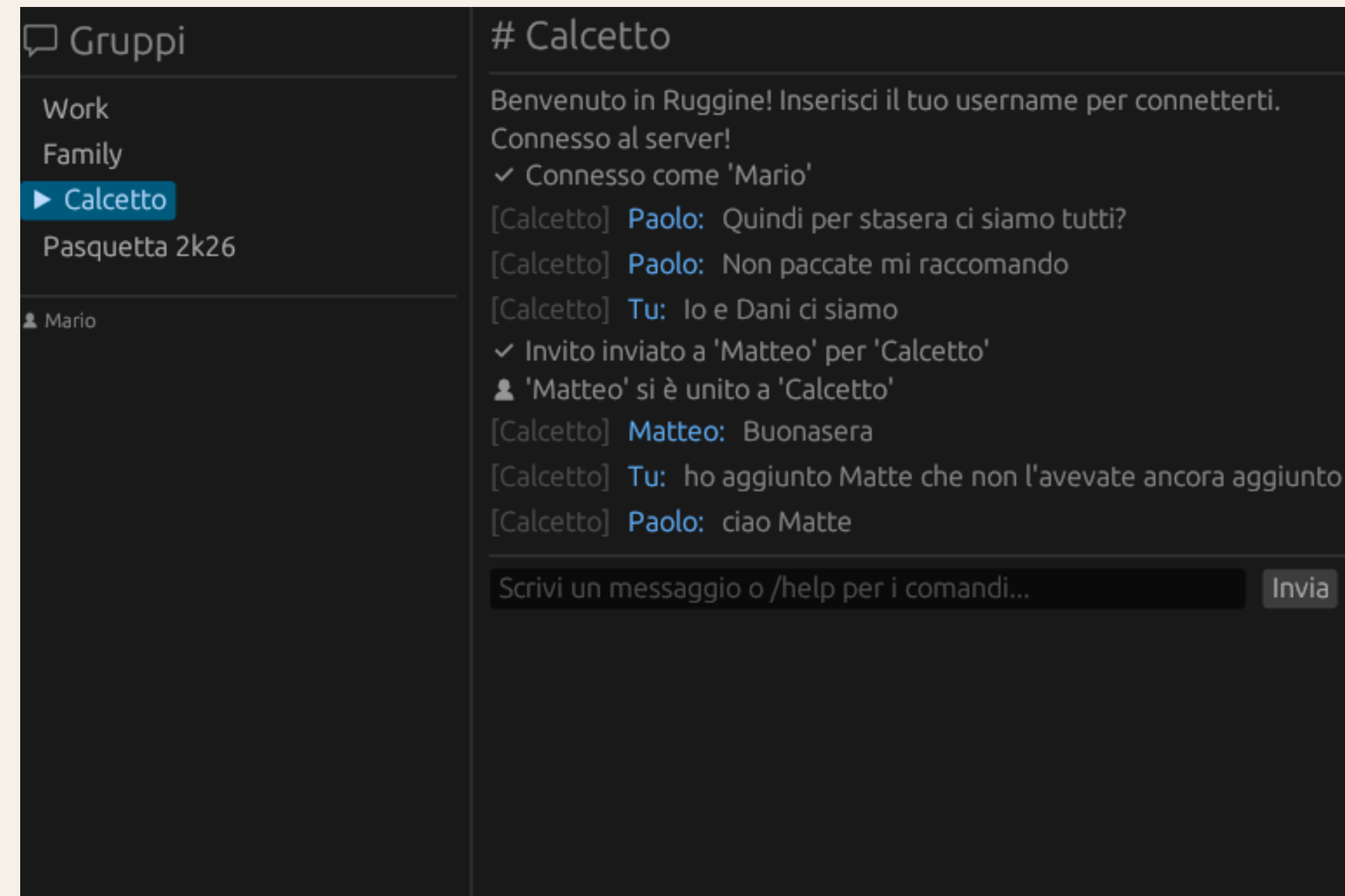
Pannello laterale: lista dei gruppi selezionabili.

Area centrale: messaggi del gruppo attivo.

Barra inferiore: input testo + pulsante Invia.

Aggiornamento real-time

`ctx.request_repaint_after(50ms)`: la GUI controlla il canale `ws_to_ui` ogni 50 millisecondi. Latenza massima di visualizzazione: 50ms.



Logging CPU

120s

Intervallo log

sysinfo

Libreria usata

< 1%

CPU overhead
del logger

cpu_logging_task()

```
async fn cpu_logging_task() {  
    let mut sys = System::new_all();  
    loop {  
        tokio::time::sleep(  
            Duration::from_secs(120)  
        ).await; // non bloccante!  
        sys.refresh_cpu();  
        let cpu = sys.global_cpu_usage(),  
            let now = Utc::now();  
            let pid = std::process::id();  
            let line = format!(  
                "[{}] PID: {} - CPU: {:.2}%\n",  
                now, pid, cpu);  
            fs::write("cpu.log", &line).ok();  
    }  
}
```

cpu.log (output)

```
[2026-03-16 15:32:27]  
PID: 26392  
CPU usage: 4.60%
```

```
[2026-03-16 15:34:27]  
PID: 27392  
CPU usage: 1.84%
```

Ottimizzazioni Build Release

opt-level = "z"

Ottimizza per dimensione minima invece che per velocità. Istruzioni più compatte, binario più piccolo.

lto = true

Link-Time Optimization: il linker analizza tutto il codice compilato e rimuove funzioni mai chiamate (dead code elimination).

codegen-units = 1

Tutto il codice compilato in un'unica unità. Consente ottimizzazioni inter-modulo più aggressive.

panic = "abort"

Invece di fare unwinding dello stack (costoso), il processo termina immediatamente. Rimuove tutto il codice di unwinding dal binario.

strip = true

Rimuove tutti i simboli di debug dal binario finale. I simboli debug sono utili in sviluppo ma inutili in produzione.

Risultato: riduzione ~40% rispetto al debug build → server 2.15 MB | client 3.36 MB

Portabilità Multi-piattaforma

Piattaforma	Architettura	Server	Client GUI	Testato
Linux (Ubuntu 22.04+)	x86_64	✓	✓	✓
Windows 10/11	x86_64	✓	✓	✓
macOS	ARM64 / x86_64	✓	✓	✓

Cross-compilation con Cargo

```
cargo build --release --target aarch64-linux-android
cargo build --release --target x86_64-pc-windows-gnu
```

Perché Rust è portable?

Compila in codice nativo per ogni target. Nessuna VM, nessun runtime richiesto. egui genera codice grafico nativo per ogni OS.

Dimensione Eseguibili

2.15 MB

ruggine_server
(Windows)

3.36 MB

ruggine_client
(Windows)

~40%

riduzione vs
debug build

Come verificare la dimensione

```
cargo build --release
```

```
# Linux / macOS
```

```
ls -lh target/release/ruggine_server
```

```
ls -lh target/release/ruggine_client
```

```
# Windows
```

```
dir target\release\ruggine_server.exe
```

File generati a runtime

ruggine.db

Database SQLite (utenti, gruppi)

cpu.log

CPU ogni 120s

logs/ruggine_server.log.*

Log applicativo rolling

Conclusioni

- ✓ Sistema client/server funzionante: chat testuale, gruppi su invito, messaggi diretti
- ✓ Implementato interamente in Rust — memory safety, zero GC, performance native
- ✓ Portabile su Linux, Windows, macOS con un unico codebase Cargo
- ✓ Concorrenza sicura: Tokio async + Arc<Mutex<>> + canali mpsc — zero race conditions
- ✓ Log CPU ogni 120s con sysinfo — consumo medio < 1% idle
- ✓ Eseguibile compatto: 2.15 MB server, 3.36 MB client (LTO + strip + opt-level=z)

Grazie per l'attenzione — Domande?